Vidyavardhaka Sangha®, Mysore
VIDYAVARDHAKA COLLEGE OF ENGINEERING
Autonomous Institute, Affiliated to Visvesvaraya Technological University, Belagavi
(Approved by AICTE, New Delhi & Government of Karnataka)
Accredited by NBA | NAAC with 'A' Grade
Department of Computer Science & Engineering
Phone: +91 821-4276230, Email: hodcs@vvce.ac.in
Web: http://www.vvce.ac.in
@vvceofficial

An activity based assessment Report

Course Name: Cloud Computing (BCSCC614)

Submitted By:

VENKATESHA Y K [4VV22CS181]

YASHWANTH S [4VV22CS189]

YESH GOWDA [4VV22CS190]

MANOJ KUMAR M [4VV23CS409]

UNDER THE GUIDANCE OF

Mr. Harsha K G

Assistant professor

Department of Computer Science & Engineering

VVCE,MYSURU

VVCE

2024-2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
VIDYAVARDHAKA COLLEGE OF ENGINEERING

Vision: The Department of Computer Science and Engineering shall create professionally competent and socially responsible engineers capable of working in global environment

Microsoft Azure in Cloud Computing

1. Account Creation and Claiming Student Free Credentials

Microsoft Azure is a powerful cloud platform offering infrastructure, platform, and software services. Students can access Azure services through the Azure for Students program, which provides $100 in free credits and access to a curated set of tools and services without requiring a credit card. The process begins by visiting the Azure for Students portal, signing up using a verified institutional email (e.g., ending with .edu or official college domain), and completing a student identity verification. Once verified, a personal Azure dashboard is created.

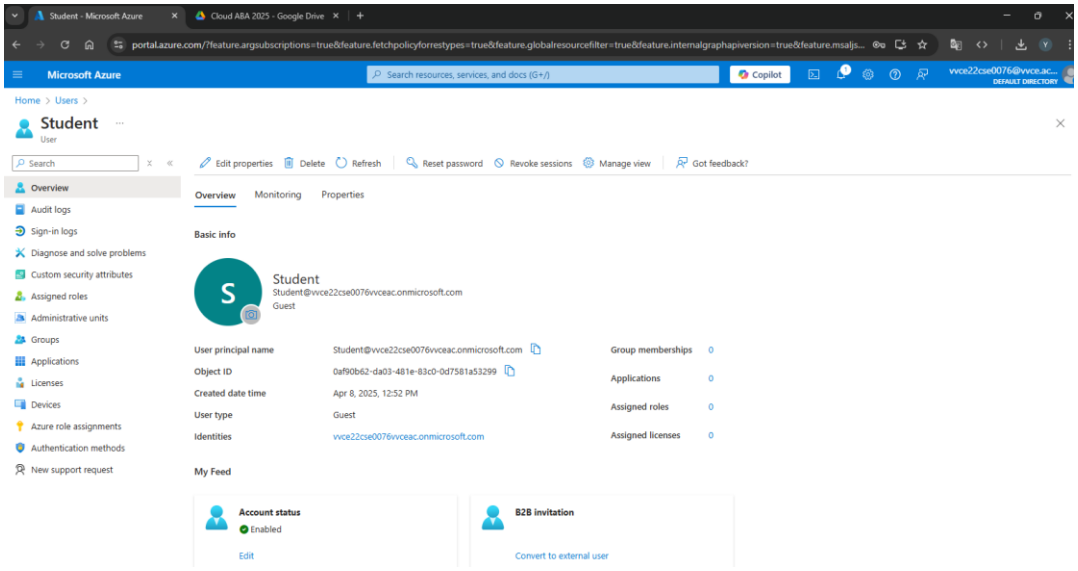


Figure: User Account Dashboard

2. Static Web App Deployment

The deployment of a **Static Web App** in Azure enables hosting static front-end files like HTML, CSS, and JavaScript directly from a GitHub repository. This is ideal for front-end developers who want serverless web hosting. The user links a GitHub repo to Azure Static Web App service, sets up a build pipeline using **GitHub Actions**, and chooses a deployment region. Every push to the GitHub main branch triggers an automatic deployment to Azure.

Steps in creating Static Web App Deployment in Microsoft Azure

Step 1: Prepare the Static Website Code

Before deployment, ensure your static website code is ready. This typically includes an index.html file and other assets such as CSS, JavaScript, and images. Push the complete project to a GitHub repository. The repository should be public or private with appropriate access permissions for deployment.

Step 2: Sign in to Azure Portal

Navigate to the Azure Portal and sign in with your Microsoft account, preferably one that has Azure for Students or other valid subscription access.

Step 3: Create a Static Web App Resource

1. In the Azure Portal, use the search bar to locate and open **"Static Web Apps"**.
2. Click on **"Create"** to initiate a new deployment.
3. Fill in the required configuration details:
    - **Subscription**: Select your Azure subscription.
    - **Resource Group**: Choose an existing group or create a new one.
    - **Name**: Provide a unique name for your Static Web App.
    - **Region**: Select the geographic region closest to your users.
    - **Deployment source**: Choose **GitHub** as the source.

Step 4: Authorize GitHub and Select Repository

You will be prompted to authorize Azure to access your GitHub account. Once authorized:

1. Select the organization (if applicable), repository, and branch you wish to deploy from.
2. Azure will use this repository as the source for deployment and set up an automated build and deployment workflow.

Step 5: Configure Build Details

Specify the following parameters to assist Azure in identifying the structure of your application:

- **App location**: Root folder of your app (e.g., / for a simple HTML project, or /src for framework-based apps).
- **API location**: Leave blank if no serverless functions are used.
- **Output location**: Directory containing the built files (e.g., build/ for React, dist/ for Angular or Vue).

Azure uses these values to configure a GitHub Actions workflow that handles the build and deployment process.

Step 6: Automated Deployment via GitHub Actions

Once setup is complete, Azure generates a GitHub Actions workflow file in your repository under .github/workflows/. This workflow automatically triggers on every code push to the selected branch. It builds the app (if applicable) and deploys the latest version to Azure Static Web Apps service.

Step 7: Access the Deployed Application

After deployment, Azure provides a globally accessible URL to your application, hosted via Azure's content delivery network (CDN). This ensures low-latency access and high availability. You can monitor and manage the deployment through the Azure Portal, where logs and configurations are available for review.

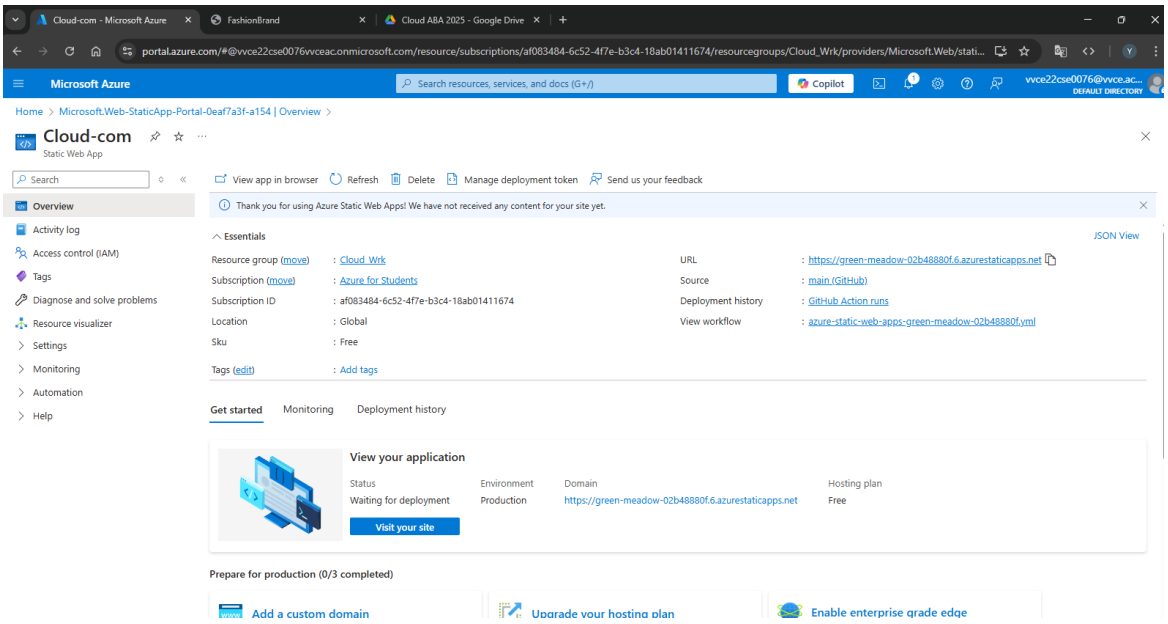


Figure: Static Web App Deployment on Azure

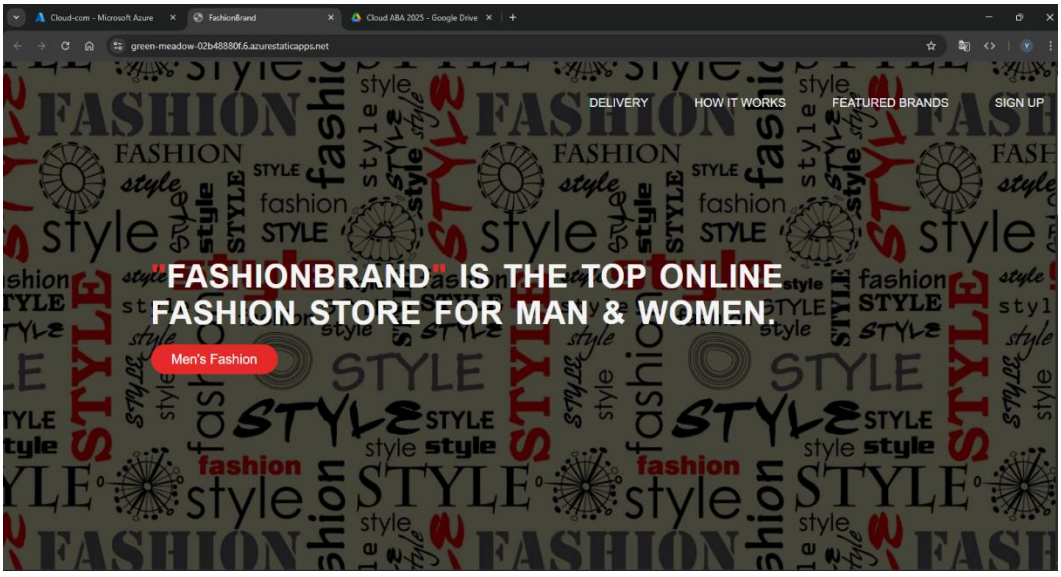


Figure: Displaying the Static Web App

3. Database Creation, Table Management & Working with Queues

Azure SQL Database is a managed relational database service based on the Microsoft SQL Server engine. Once a database is created, users can connect through tools like SSMS or Azure Data Studio to execute SQL queries and define tables.

In this case, a Students table was created to store records using SQL DDL (Data Definition Language) queries. Simultaneously, Azure Queue Storage was used to implement message queuing for asynchronous communication between components of a distributed application. A Python/Azure Function app was used to enqueue and dequeue messages.

A. Creating an Azure SQL Database

1. Log in to the Azure Portal
    - Go to https://portal.azure.com and sign in.
2. Create a New SQL Database
    - In the search bar, type "SQL Database" and select Create.
    - Configure the following:
        - Database name (e.g., StudentDB)
        - Resource group: Create or select one.
        - Server: Create a new SQL Server instance with:
            - Unique server name (e.g., myazuresqlserver)
            - Admin login and password
            - Choose a region
        - Compute + Storage: Use the basic tier (for free account compatibility).
3. Configure Networking
    - Allow Azure services to access the server.
    - Add your client IP to the firewall rules to access the database from your local machine.
4. Review and Create
    - Click Review + Create, then Create.
    - Deployment takes a few minutes.

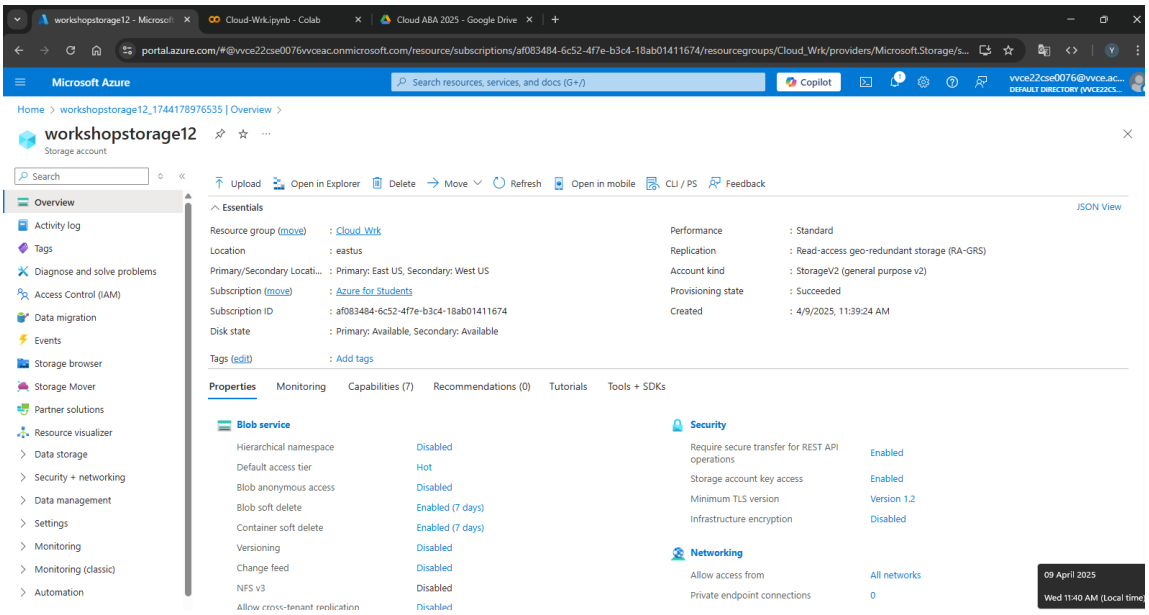


Figure: Cloud Database Overview

B. Creating Tables in the SQL Database

1. Access the SQL Database
    - Go to the SQL databases section.
    - Select your database and click Query editor (preview).
2. Authenticate with SQL Server Admin credentials.
3. Run SQL Query to Create Table

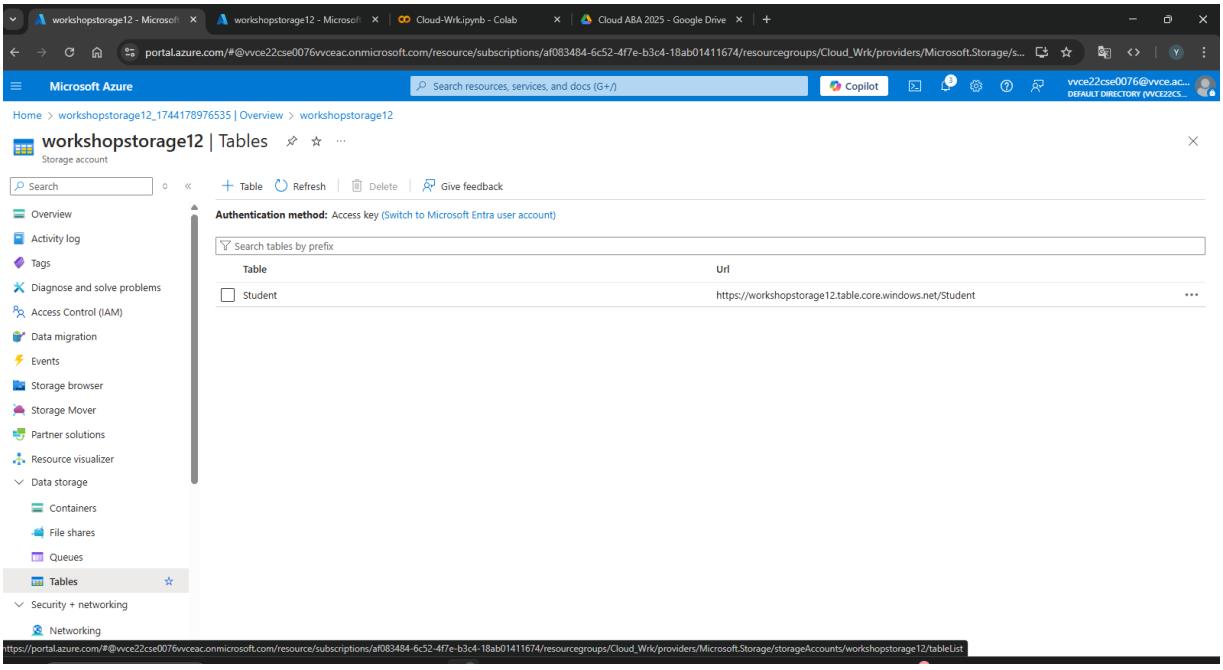


Figure: Cloud Database Table Creation

C. Working with Azure Storage Queues

1. Create a Storage Account
    - In the Azure Portal, search for Storage Accounts > Create.
        - Storage account name (e.g., mystoragequeue)
        - Region and performance settings
        - Redundancy: Use locally redundant storage (LRS)
2. Add a Queue
    - After deployment, navigate to the storage account.
    - Under Data Storage, select Queues.
    - Click + Queue, and name it (e.g., taskqueue).

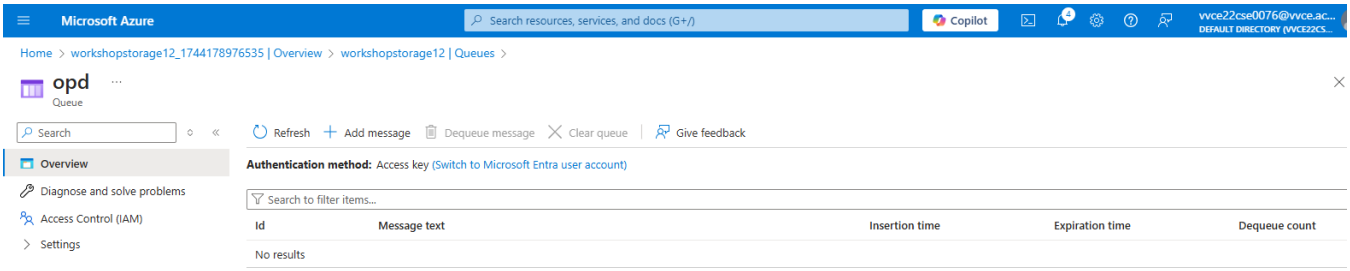


Figure: Cloud Database Queue Dashboard

4. Creating a Key using Azure Key Vault

Azure Key Vault is a cloud service that provides secure storage and access to secrets, encryption keys, and certificates. After creating a Key Vault, the user generates a cryptographic key, which is used for encryption operations such as securing data tokens, credentials, or confidential documents.

Access policies are set to define which apps or users can access this key. Applications can retrieve these secrets or keys via secure REST APIs or SDKs without exposing the key in code.

Steps to Create a Key in Azure Key Vault

Step 1: Sign in to Azure Portal

- Visit https://portal.azure.com
- Log in with your Microsoft credentials.

Step 2: Create a New Key Vault

1. In the search bar, type "Key Vault" and select Key Vaults.
2. Click Create.
3. Fill in the required details:
    - Subscription: Select your subscription (e.g., Azure for Students).
    - Resource Group: Use an existing group or create a new one.
    - Key Vault Name: Provide a unique name (e.g., MySecureVault).
    - Region: Choose the appropriate region (e.g., Central India).
4. Under Pricing tier, keep the default (Standard).
5. Click Review + Create, then Create.

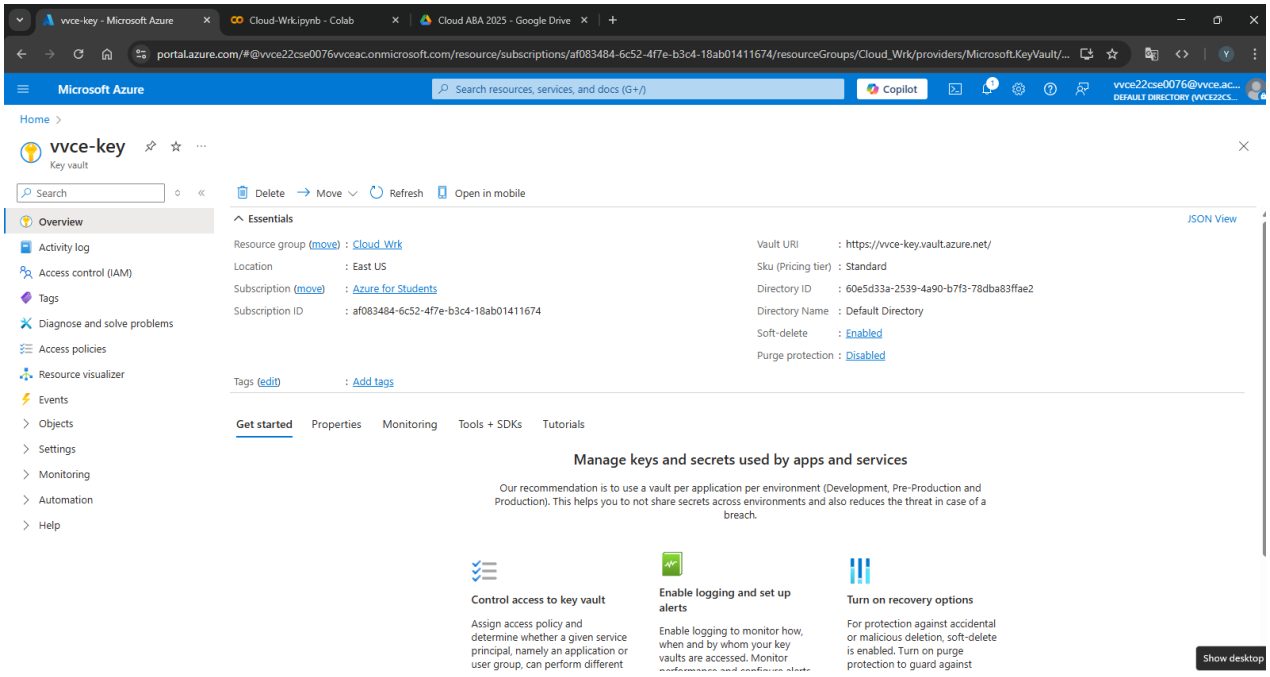


Figure: Creating a Dummy Key

5. Building a Translator using Azure Cognitive Services

The **Azure Translator API**, part of Azure Cognitive Services, provides real-time translation capabilities between more than 70 languages. A Translator resource is provisioned, and its API credentials are used to make HTTP POST requests with text inputs and target languages.

A web or mobile application can be integrated with this API using REST or SDKs (JavaScript, Python, etc.), enabling language translation features for global users.

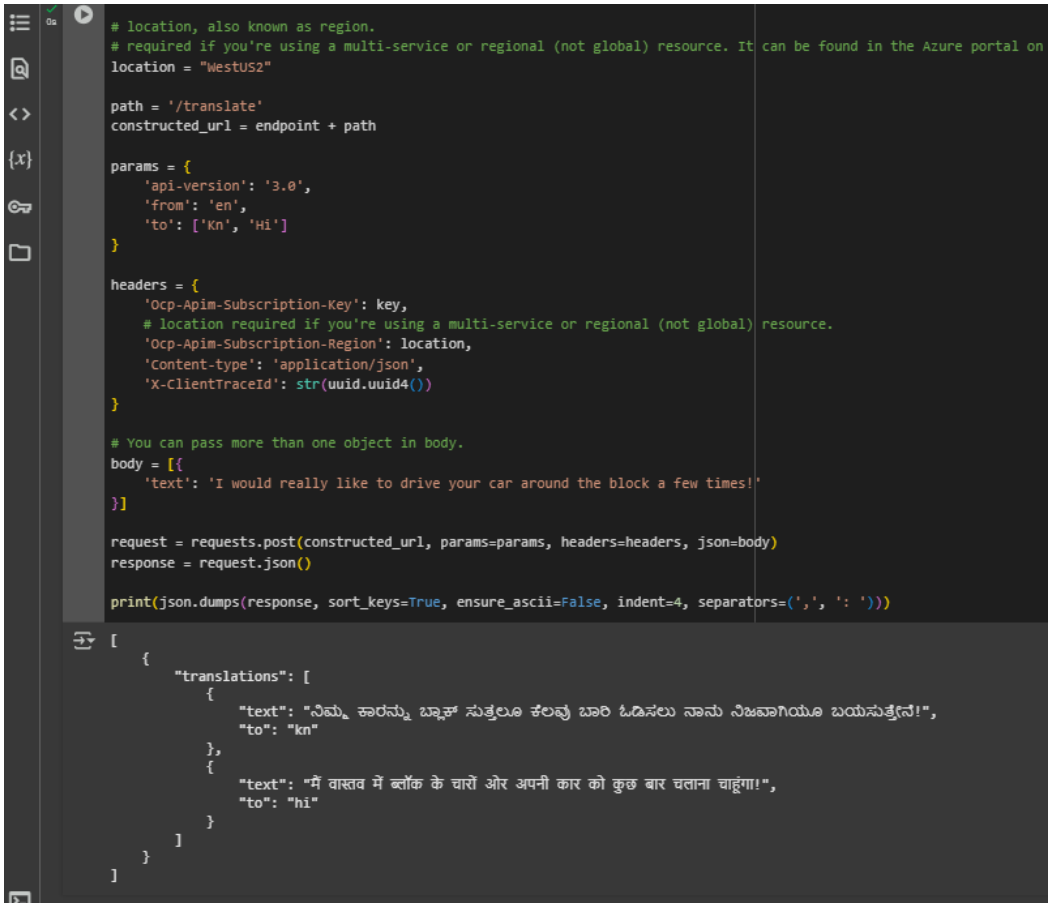
Steps to Build and Integrate Azure Translator

Step 1: Sign in to Azure Portal

- Go to https://portal.azure.com
- Sign in with your Microsoft account.

Step 2: Create a Translator Resource

1. In the search bar, type “Translator” and select Translator (Cognitive Services).
2. Click Create.
3. Fill in the required details:
    - Subscription: Select your subscription (e.g., Azure for Students).
    - Resource Group: Choose or create a new one.
    - Region: Select an appropriate region (e.g., Central India).
    - Name: Provide a unique name (e.g., MyTranslatorAPI).
    - Pricing tier: Use the free tier F0 (suitable for development/testing).
4. Click Review + Create, then Create.



```
# location, also known as region.
# required if you're using a multi-service or regional (not global) resource. It can be found in the Azure portal on
location = "WestUS2"

path = '/translate'
constructed_url = endpoint + path

params = {
    'api-version': '3.0',
    'from': 'en',
    'to': ['Kn', 'Hi']
}

headers = {
    'Ocp-Apim-Subscription-Key': key,
    # location required if you're using a multi-service or regional (not global) resource.
    'Ocp-Apim-Subscription-Region': location,
    'Content-type': 'application/json',
    'X-ClientTraceId': str(uuid.uuid4())
}

# You can pass more than one object in body.
body = [{
    'text': 'I would really like to drive your car around the block a few times!'
}]

request = requests.post(constructed_url, params=params, headers=headers, json=body)
response = request.json()

print(json.dumps(response, sort_keys=True, ensure_ascii=False, indent=4, separators=(',', ': ')))
```

```
[
    {
        "translations": [
            {
                "text": "ನಿಮ್ಮ ಕಾರನ್ನು ಬ್ಲಾಕ್ ಸುತ್ತಲೂ ಕೆಲವು ಬಾರಿ ಓಡಿಸಲು ನಾನು ನಿಜವಾಗಿಯೂ ಬಯಸುತ್ತೇನೆ!",
                "to": "kn"
            },
            {
                "text": "मैं वास्तव में ब्लॉक के चारों ओर अपनी कार को कुछ बार चलाना चाहूंगा!",
                "to": "hi"
            }
        ]
    }
]
```

Figure: Running Translation Code on Google Collab

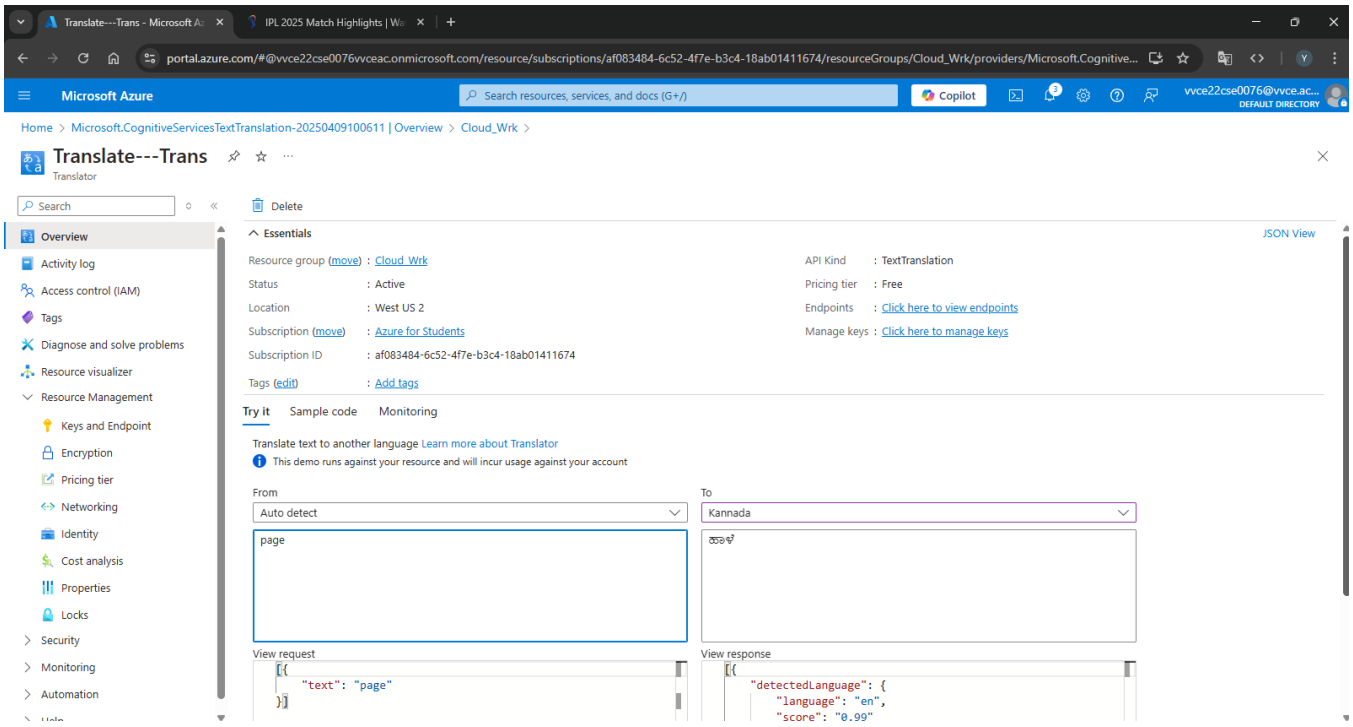


Figure: Testing the Translator on Azure Platform

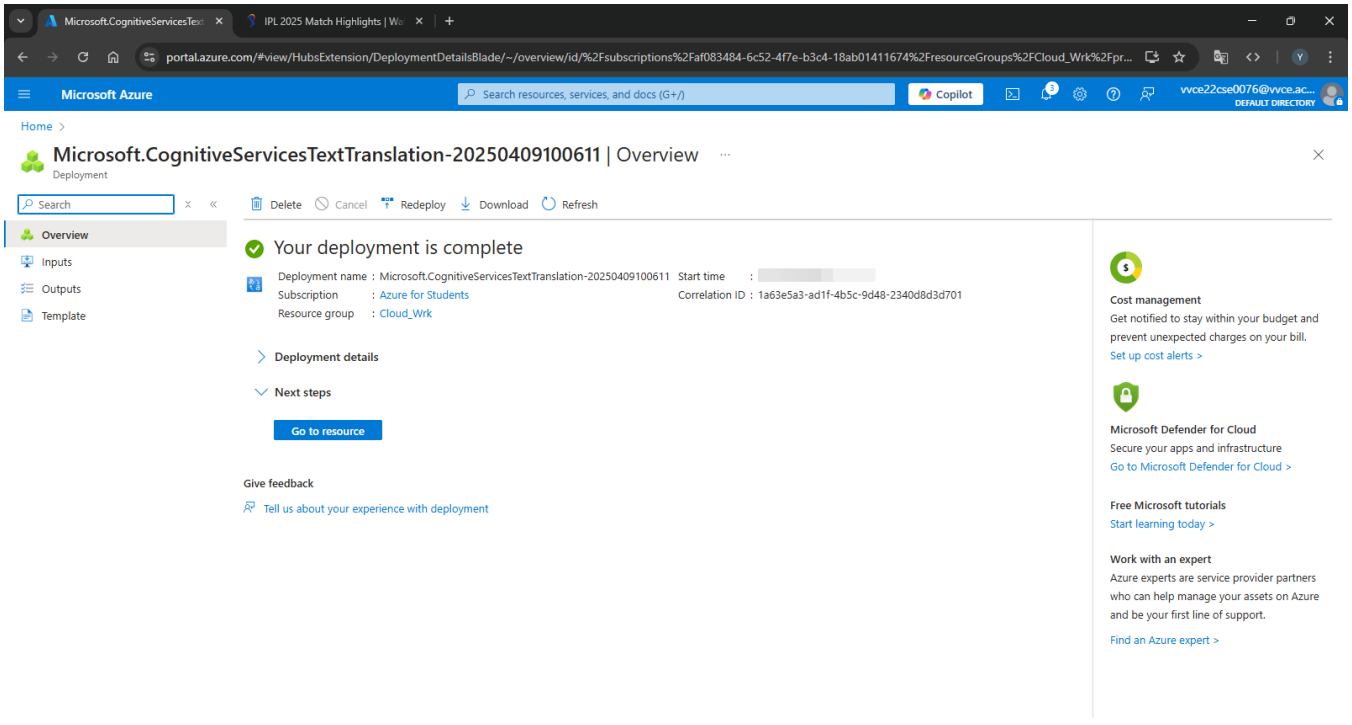


Figure: Building a Translator on Azure

6. Virtual Machine Creation and RDP File Connection

Azure Virtual Machines (VMs) provide Infrastructure as a Service (IaaS), enabling users to run Windows or Linux operating systems in the cloud. A Windows Server 2022 VM was created with B1s (Basic Tier) sizing. During the setup, the user downloads an **RDP (Remote Desktop Protocol)** file, which allows remote desktop access to the VM from a local machine.

Users can perform software installations, test applications, or run development environments as they would on a physical machine.

Steps to Create a Windows VM and Connect Using RDP

Step 1: Sign in to Azure Portal

- Navigate to https://portal.azure.com
- Log in with your Microsoft account (e.g., Azure for Students credentials)

Step 2: Create a Virtual Machine

1. From the Azure Dashboard, click “Create a resource”.
2. Choose “Virtual Machine” from the Compute category.
3. Fill in the required Basic Details:
    - Subscription: Azure for Students or your assigned subscription.
    - Resource Group: Select an existing one or create a new group.
    - Virtual Machine Name: e.g., MyWindowsVM
    - Region: Choose closest region (e.g., Central India).
    - Availability options: Leave default (No infrastructure redundancy required).
    - Image: Select Windows Server 2022 Datacenter.
    - Size: Choose B1s (1 vCPU, 1 GiB RAM) – part of free tier.

Step 3: Configure Administrator Account

- Username: e.g., azureuser
- Password: Set a strong password (8+ characters, includes upper, lower, number, special character)
- Ensure Inbound port rules has:
    - Public inbound ports: Allow selected ports
    - Select inbound ports: RDP (3389)

Step 4: Disk and Network Settings (Default)

- Leave default settings for Disks (Standard SSD) and Networking
- Click Next for each section or Review + Create

Step 5: Review and Create

- Review the configuration
- Click Create
- The deployment will begin and complete in ~2–3 minutes

Step 6: Connect to the Virtual Machine Using RDP

1. After deployment, go to the Virtual Machines panel and select your VM.
2. Click “Connect” → RDP.
3. Azure will generate an .rdp file. Click Download RDP File.
4. Open the downloaded .rdp file on your local Windows system.
5. A Remote Desktop Connection window will appear:
    - Enter the username: azureuser
    - Enter the password you created earlier
6. Accept the certificate prompt and proceed.

Step 7: Use the Virtual Machine

Once connected:

- You will see a full remote desktop interface of Windows Server 2022.
- You can:
    - Install software
    - Browse using Internet Explorer (Edge)
    - Deploy and test applications
    - Work as if on a personal computer

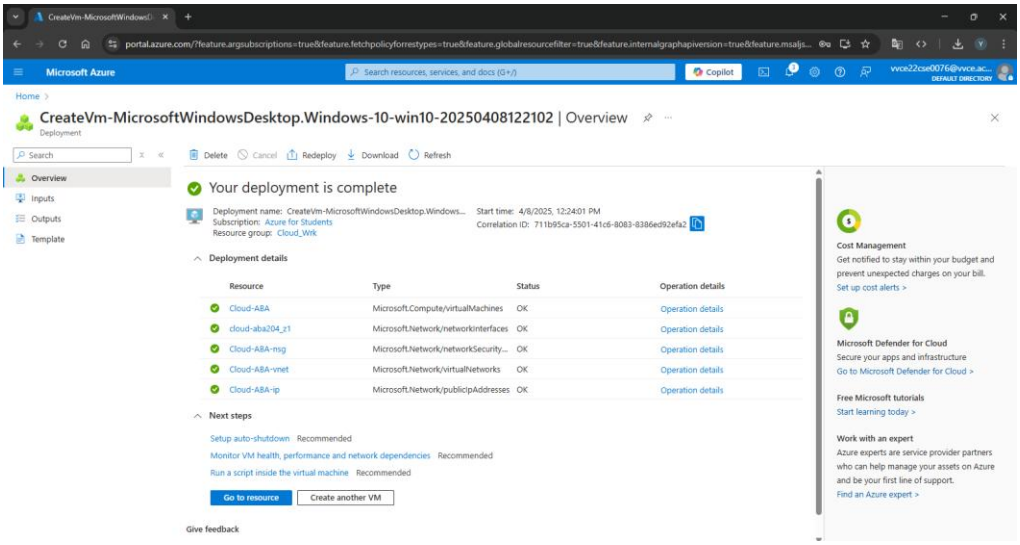


Figure: Creating a VM on Azure

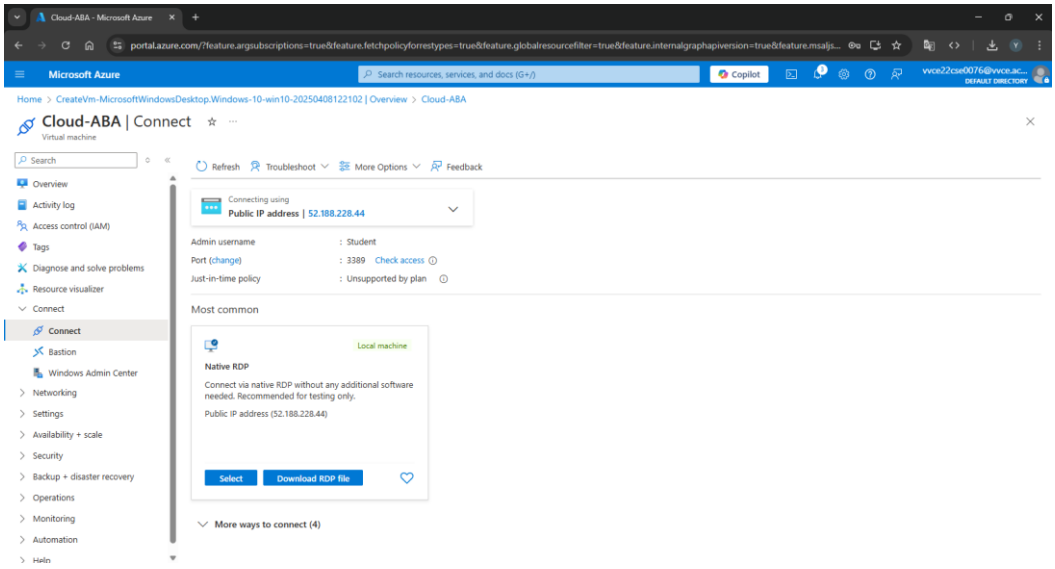


Figure: Running a VM on Azure

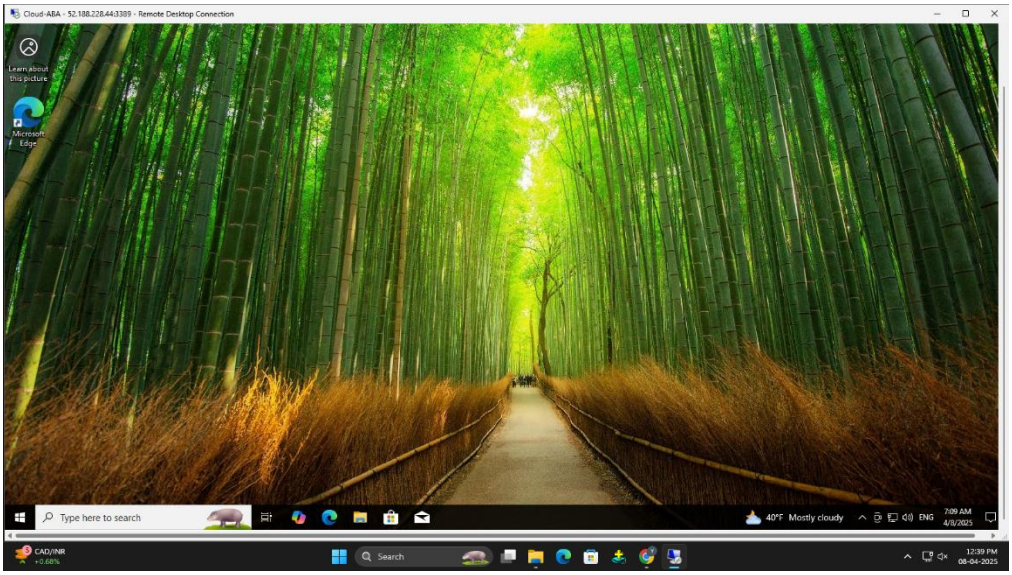


Figure: Connect to the Virtual Machine Using RDP